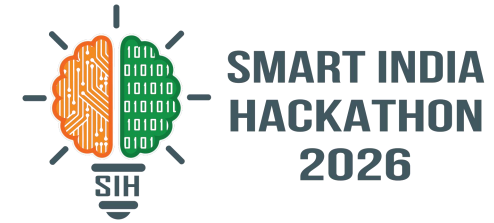


SMART INDIA HACKATHON 2026



TITLE PAGE

Problem Statement ID – 26126

Problem Statement Title – Vision Based Autonomous Navigation for Unmanned Ground Vehicle for Outdoor environment

Theme – Smart Automation

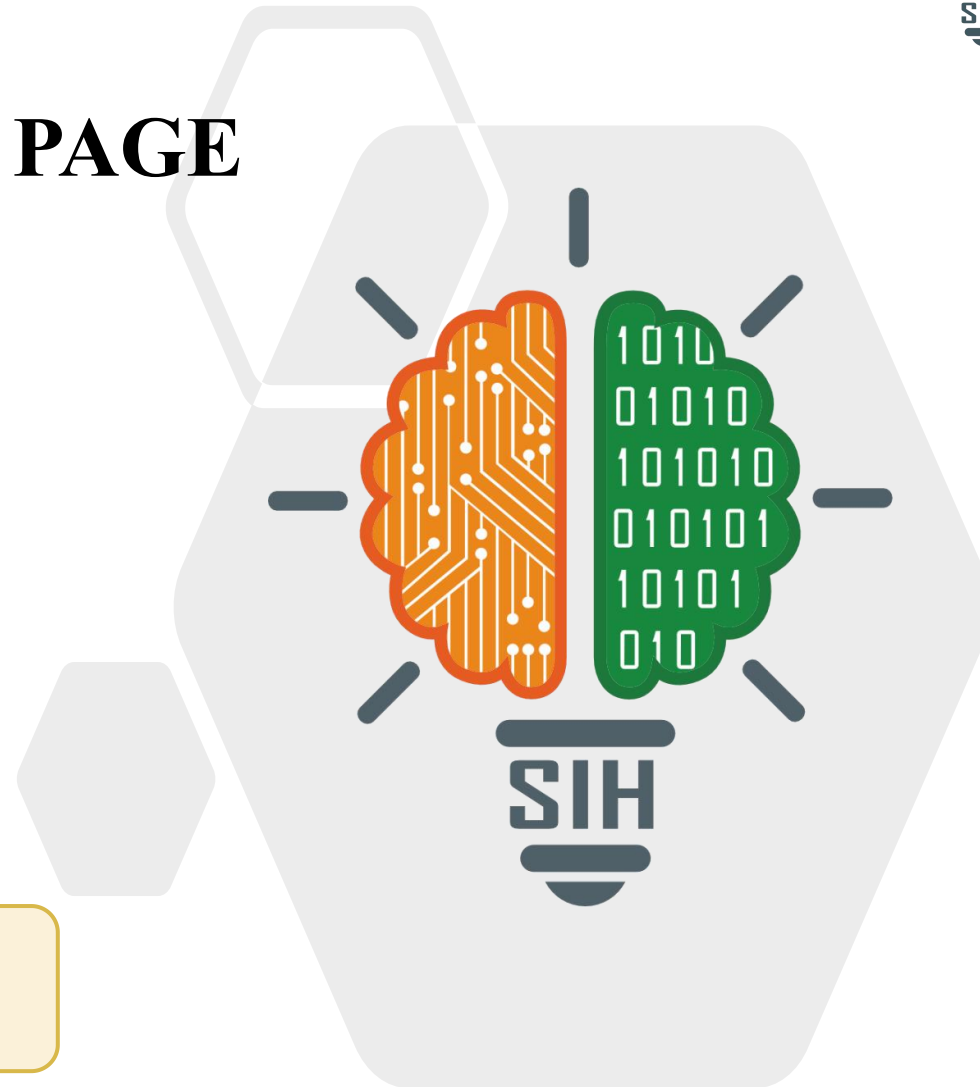
PS Category – Software

Team ID – <Team ID>

Team Name – The Vikings

SIH 2025 GRAND FINALE FINALIST

Team The Vikings — returning to Smart India Hackathon 2026



IDEA TITLE

DRISHTI-UGV — a vision-first autonomy module that drives an outdoor UGV from Point A to Point B with no GPS

THE PROBLEM

01**GPS is denied or unreliable**

Tree cover, terrain masking and jamming remove any trusted fix.

02**Open ground is not safe ground**

A ditch, mud patch or slope still reads as free space.

03**Obstacles appear after planning**

People, animals and vehicles cross the path mid-mission.

04**Vision degrades without warning**

Glare, shadow and low light silently break a camera-only stack.

OUR SOLUTION

A**Visual SLAM replaces GPS**

RTAB-Map fuses stereo and IMU into a drift-bounded pose.

B**Terrain is scored, not just occupied**

Slope, roughness and height variance become real driving cost.

C**Semantics fused into the costmap**

YOLO marks mud, water, rock and people; unknown is priced high.

D**Safety supervisor outside the AI**

Deterministic code halts on stale sensors or lost pose.

INNOVATION & UNIQUENESS

Deterministic safety, not learned

The stop decision never depends on model confidence.

Camera-first, LiDAR-free terrain

Traversability from vision geometry plus semantics.

Proven by simulation at scale

Randomised Isaac Sim missions score it before hardware.

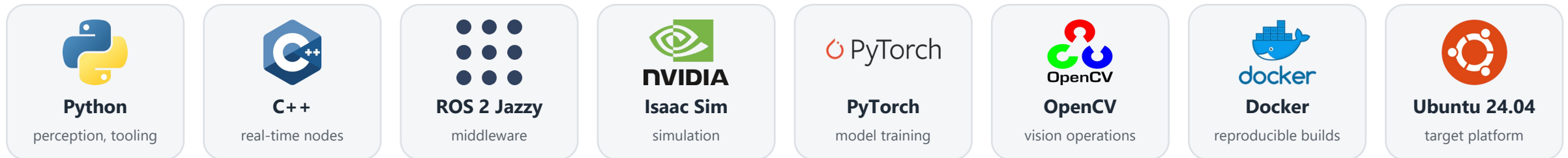
TECHNICAL APPROACH

PROCESSING PIPELINE — camera in, safe velocity out



Output: /cmd_vel to the UGV base • every block is a ROS 2 node behind a fixed topic and TF contract, so the identical graph runs in simulation and on the real vehicle

LANGUAGES, FRAMEWORKS & TOOLS



IMPLEMENTATION METHODOLOGY — simulation-first; every phase ships a runnable artefact



Design principle: the neural network only answers “what am I looking at?”

Coordinate transforms, mapping, planning, control and the stop decision stay in deterministic, auditable code — so a perception failure degrades into a safe halt, never a collision.

FEASIBILITY AND VIABILITY

F

FEASIBLE TODAY

- Nav2, RTAB-Map and elevation_mapping_cupy are mature ROS 2 packages
- Integration and tuning — nothing has to be invented
- Simulation-first: development starts with zero hardware
- Whole loop runs on one RTX workstation (32 GB RAM, 16 GB VRAM)

R

KEY RISKS

- TF, timestamp and calibration errors — the top failure cause
- SLAM drift on featureless or low-texture terrain
- A ditch or water surface misread as drivable ground
- Simulation-to-reality gap in lighting and texture

M

MITIGATION

- Validate the TF tree and clock sync before any AI work
- Stereo + IMU fusion, speed limits, automatic re-localisation
- Unknown terrain is priced expensive, never free
- Domain randomisation over light, texture, noise and obstacles

V

VIABILITY BEYOND SIH

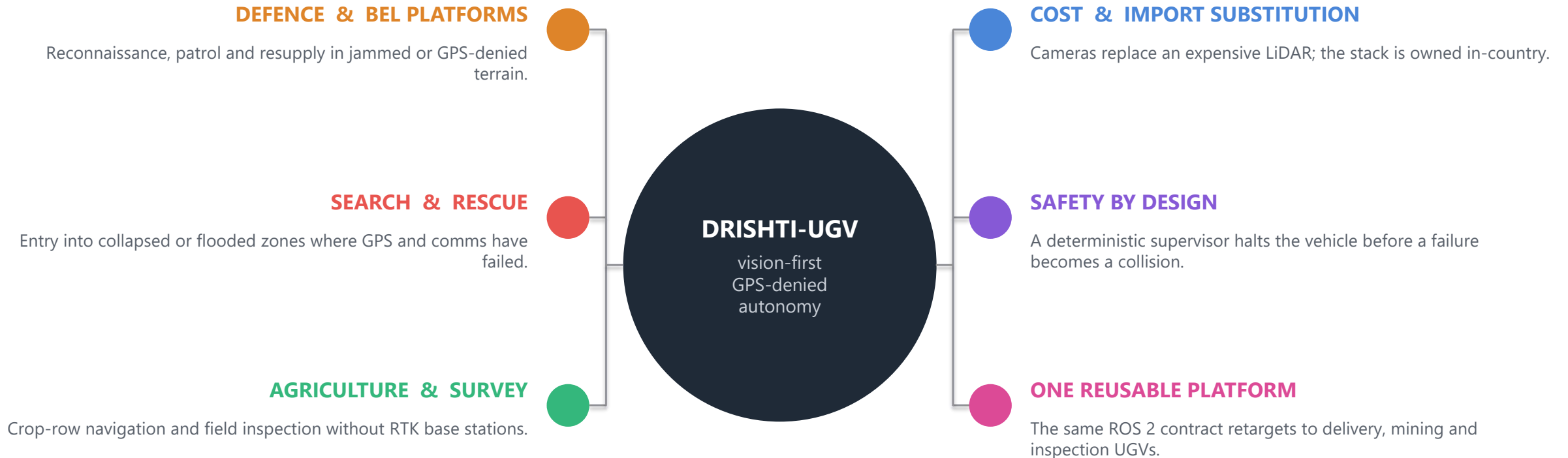
- Identical ROS 2 topic and TF contract, simulation and vehicle
- Only sensor driver and motor interface change on hardware
- Profiles down to a Jetson-class edge computer
- Reusable across BEL surveillance and logistics platforms

ACCEPTANCE GATES — what we will measure, instead of claiming “100% accuracy”

≥ 95%collision-free
randomised missions**< 2%**localisation drift
over distance travelled**< 200 ms**emergency-stop
software response**≤ 100 ms**perception
latency**≥ 20 Hz**control loop
rate**≥ 97%**goal completion
rate

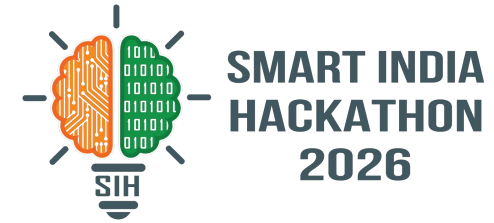
Every gate is scored automatically from rosbag2 logs against Isaac Sim ground truth, so the numbers are reproducible rather than asserted.

IMPACT AND BENEFITS

**Measured, not claimed.**

Every benefit above is backed by a mission-level number — collision-free completion rate, localisation error against simulator ground truth, path efficiency, recovery rate and emergency-stop latency.

RESEARCH AND REFERENCES



NAVIGATION & CONTROL

- `ros-navigation / navigation2` — planners, costmaps, behaviour trees
- Nav2 MPPI controller — sampling-based predictive local control
- `sacrover / traversability-nav2` — outdoor traversability costmap

VISUAL LOCALISATION & SLAM

- `introlab / rtabmap_ros` — ROS 2 stereo and RGB-D SLAM with Nav2
- `UZ-SLAMLab / ORB_SLAM3` — visual-inertial benchmark (GPLv3)
- `LARIAD / Offroad-Nav` — closest end-to-end off-road reference

TERRAIN & PERCEPTION

- `leggedrobotics / elevation_mapping_cupy` — GPU elevation layers
- `ultralytics` — detection and segmentation models (licence reviewed)
- `DepthAnything / Depth-Anything-V2` — monocular depth fallback

SOURCE LINKS & OFFICIAL DOCUMENTATION

- › Nav2 framework — github.com/ros-navigation/navigation2
- › RTAB-Map ROS 2 — github.com/introlab/rtabmap_ros
- › Elevation Mapping CuPy — github.com/leggedrobotics/elevation_mapping_cupy
- › Traversability Nav2 — github.com/sacrover/traversability-nav2
- › Offroad-Nav reference — github.com/LARIAD/Offroad-Nav

- › Isaac Sim ROS 2 bridge — docs.isaacsim.omniverse.nvidia.com
- › Isaac Sim workspaces — github.com/isaac-sim/IsaacSim-ros_workspaces
- › Ultralytics YOLO — github.com/ultralytics/ultralytics
- › Depth Anything V2 — github.com/DepthAnything/Depth-Anything-V2
- › Open3D — github.com/isl-org/Open3D

MIT / BSD / Apache-2.0

Nav2, RTAB-Map, `elevation_mapping_cupy`, `Offroad-Nav`, `Open3D`

Licence-reviewed before use

`Ultralytics` and `Depth Anything V2 Small` — terms checked

Excluded from the shipped build

`ORB-SLAM3 (GPLv3)` used only as an offline benchmark

Licence position screened per component before integration; every third-party notice is preserved in the delivered repository.